

Functional Programming

Lecture 2

Jesper Bengtson

Based on original slides by Michael R. Hansen & Patrick Bahr

Last week

We introduced functional programming

- functions are first-class citizens
- Keep side effects to a minimum
- type inference vs type checking
- recursive functions rather than loops

Questions?

This week

- I. Expression evaluation
- II. Function definitions
- III. Lists
- IV. User-defined types

Everything we do from now on depends on this and next week's lecture.

Come back to it often. Study it often.

Do not wait until the exam to catch up on things you might have missed.

Part I

Expressions

Statements and expressions

Imperative languages are typically structured around
statements and **expressions**

Statements and expressions

Imperative languages are typically structured around
statements and **expressions**

Statements

- do things (assign values to variables, print to screen, update memory, ...)
- are built using statements and expressions

Statements and expressions

Imperative languages are typically structured around **statements** and **expressions**

Statements

- do things (assign values to variables, print to screen, update memory, ...)
- are built using statements and expressions

Expressions

- are evaluated to values
(e.g. $2 + 5 \leadsto 7$)
- are built using expressions only

Statements and expressions

Imperative languages are typically structured around statements and expressions

Java

```
y = 4;  
if (x > y) {  
    return "gt4";  
}  
else {  
    return "lt4";  
}
```

Statements and expressions

Imperative languages are typically structured around statements and expressions

Java

```
y = 4;  
if (x > y) {  
    return "gt4";  
}  
else {  
    return "lt4";  
}
```

This entire block of code is a statement consisting of an assignment and a conditional

Statements and expressions

Imperative languages are typically structured around statements and expressions

Java

```
y = 4;
```

```
if (x > y) {  
    return "gt4";  
}  
else {  
    return "lt4";  
}
```

The assignment is a statement that assigns the value 4 to the variable *y*

Statements and expressions

Imperative languages are typically structured around statements and expressions

Java

```
y = 4;  
if (x > y) {  
    return "gt4";  
}  
else {  
    return "lt4";  
}
```

This if-statement is a statement that either proceeds with the 'then branch' (return "gt4") or the 'else branch' (return "lt4")

Statements and expressions

Imperative languages are typically structured around statements and expressions

Java

```
y = 4;  
if (x > y) {  
    return "gt4";  
}  
else {  
    return "lt4";  
}
```

The return statement exits the current method and sets the return value to "gt4"

Statements and expressions

Imperative languages are typically structured around statements and expressions

Java

```
y = 4;  
if (x > y) {  
    return "gt4";  
}  
else {  
    return "lt4";  
}
```

The return statement exits the current method and sets the return value to "lt4"

Statements and expressions

Imperative languages are typically structured around statements and expressions

Java

```
y = 4;  
if (x > y) {  
    return "gt4";  
}  
else {  
    return "lt4";  
}
```

4 is an expression that evaluates to the integer 4

Statements and expressions

Imperative languages are typically structured around statements and expressions

Java

```
y = 4;  
if (x > y) {  
    return "gt4";  
}  
else {  
    return "lt4";  
}
```

$x > y$ is an expression that evaluates to `true` if `x` is strictly greater than `y` and `false` otherwise

Statements and expressions

Imperative languages are typically structured around statements and expressions

Java

```
y = 4;  
if (x > y) {  
    return "gt4";  
}  
else {  
    return "lt4";  
}
```

"gt4" is an expression that evaluates to the string "gt4"

Statements and expressions

Imperative languages are typically structured around statements and expressions

Java

```
y = 4;  
if (x > y) {  
    return "gt4";  
}  
else {  
    return "lt4";  
}
```

"lt4" is an expression that evaluates to the string "lt4"

Statements and expressions

Functional languages **only** use expressions

Java

```
y = 4;  
if (x > y) {  
    return "gt4";  
}  
else {  
    return "lt4";  
}
```

F#

```
let y = 4  
if x > y then  
    "gt4"  
else  
    "lt4"
```


Statements and expressions

Functional languages **only** use expressions

This entire block of code is a an expression that evaluates to the string "gt4" if $x > 4$ and "lt4" otherwise

F#

```
let y = 4
if x > y then
    "gt4"
else
    "lt4"
```


Statements and expressions

Functional languages **only** use expressions

This if-statement is a an expression that returns the string "gt4" if $x > 4$ and "lt4" otherwise

F#

```
let y = 4
if x > y then
    "gt4"
else
    "lt4"
```

Statements and expressions

Functional languages **only** use expressions

4 is an expression that evaluates to the integer 4

F#

```
let y = 4
if x > y then
    "gt4"
else
    "lt4"
```

Statements and expressions

Functional languages **only** use expressions

$x > y$ is an expression that evaluates to `true` if `x` is strictly greater than `y` and `false` otherwise

No parentheses needed!

F#

```
let y = 4
if x > y then
    "gt4"
else
    "lt4"
```


Statements and expressions

Functional languages **only** use expressions

"gt4" is an expression that evaluates to the string "gt4"

F#

```
let y = 4
if x > y then
    "gt4"
else
    "lt4"
```


Statements and expressions

Functional languages **only** use expressions

"1t4" is an expression that evaluates to the string "1t4"

F#

```
let y = 4
if x > y then
    "gt4"
else
    "1t4"
```

Statements and expressions

Expressions evaluate to a single value

```
let y = 4
if x > y then
  "gt4"
else
  "lt4"
```

Statements and expressions

Expressions evaluate to a single value

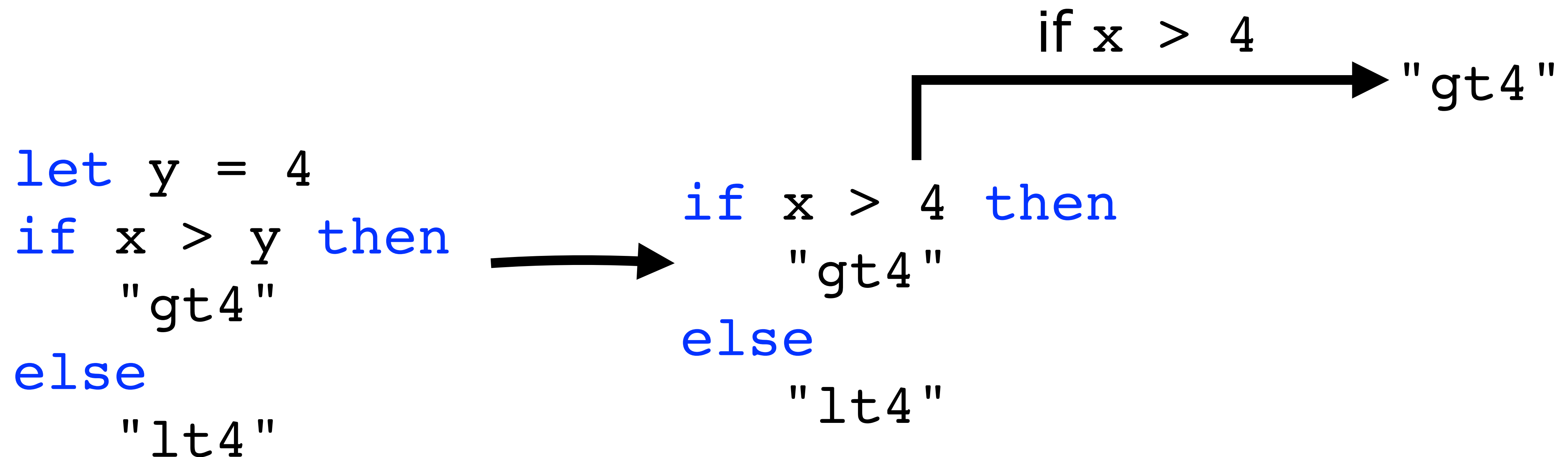
```
let y = 4
if x > y then
  "gt4"
else
  "lt4"
```



```
if x > 4 then
  "gt4"
else
  "lt4"
```

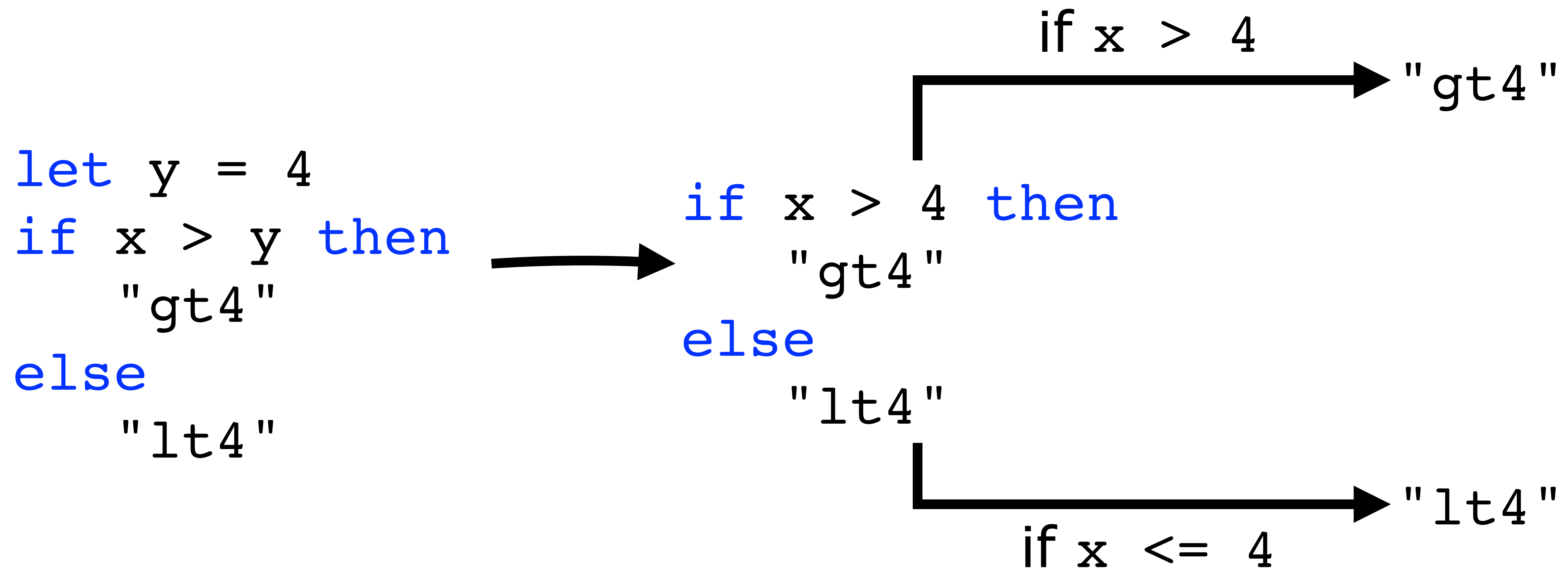

Statements and expressions

Expressions evaluate to a single value



Statements and expressions

Expressions evaluate to a single value



Statements and expressions

Assuming that `x` has type `int`, what is the type of this expression?

```
let y = 4
if x > y then
  "gt4"
else
  "lt4"
```

- A. `int`
- B. `string`
- C. `int -> string`
- D. `int -> int -> string`

Statements and expressions

Is this possible in either language?

Java

```
"result is: " +  
y = 4;  
if (x > y) {  
    return "gt4";  
}  
else {  
    return "lt4";  
}
```

F#

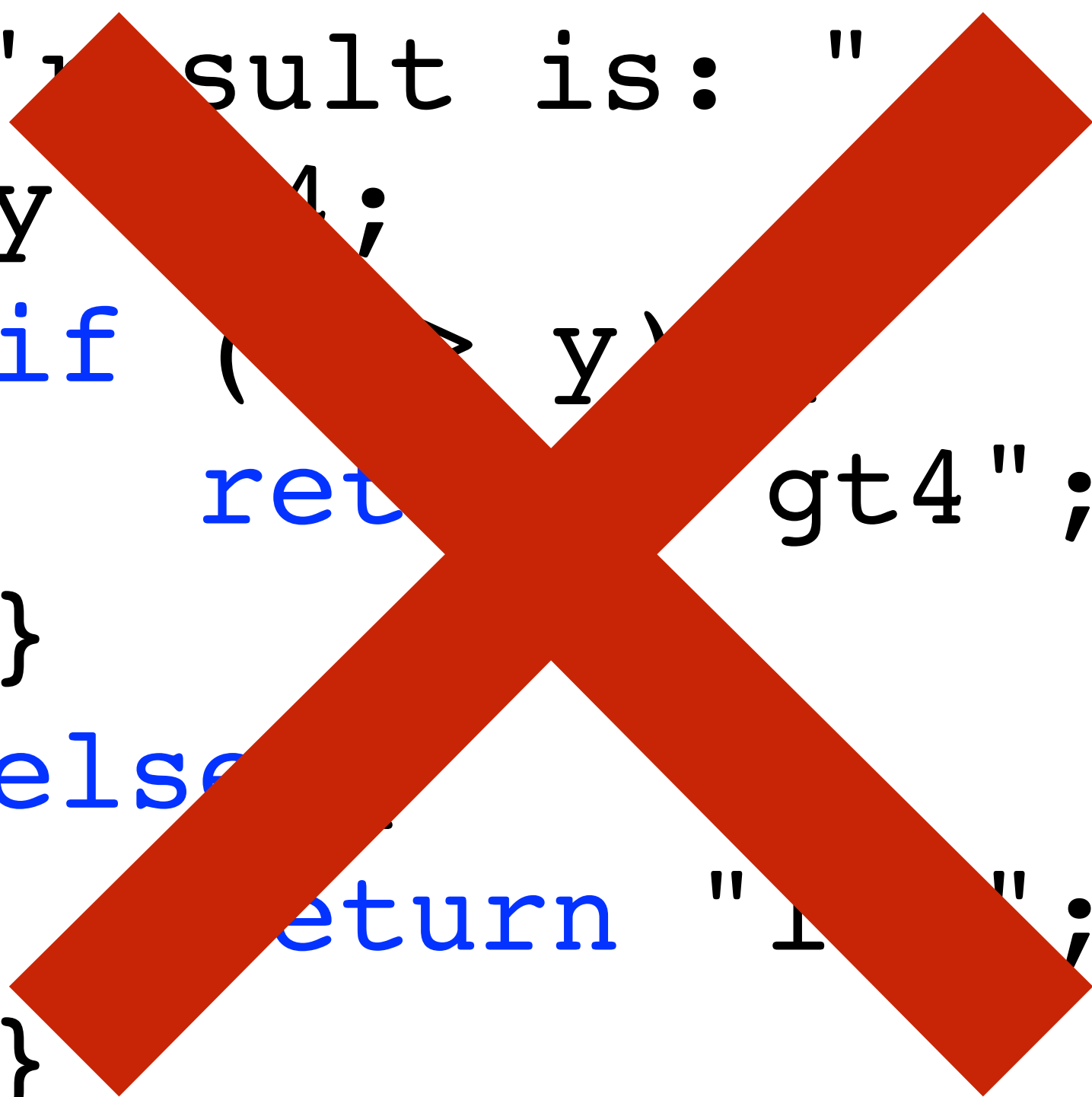
```
"result is: " +  
let y = 4  
if (x > y) then  
    "gt4"  
else  
    "lt4"
```


Statements and expressions

Is this possible in either language?

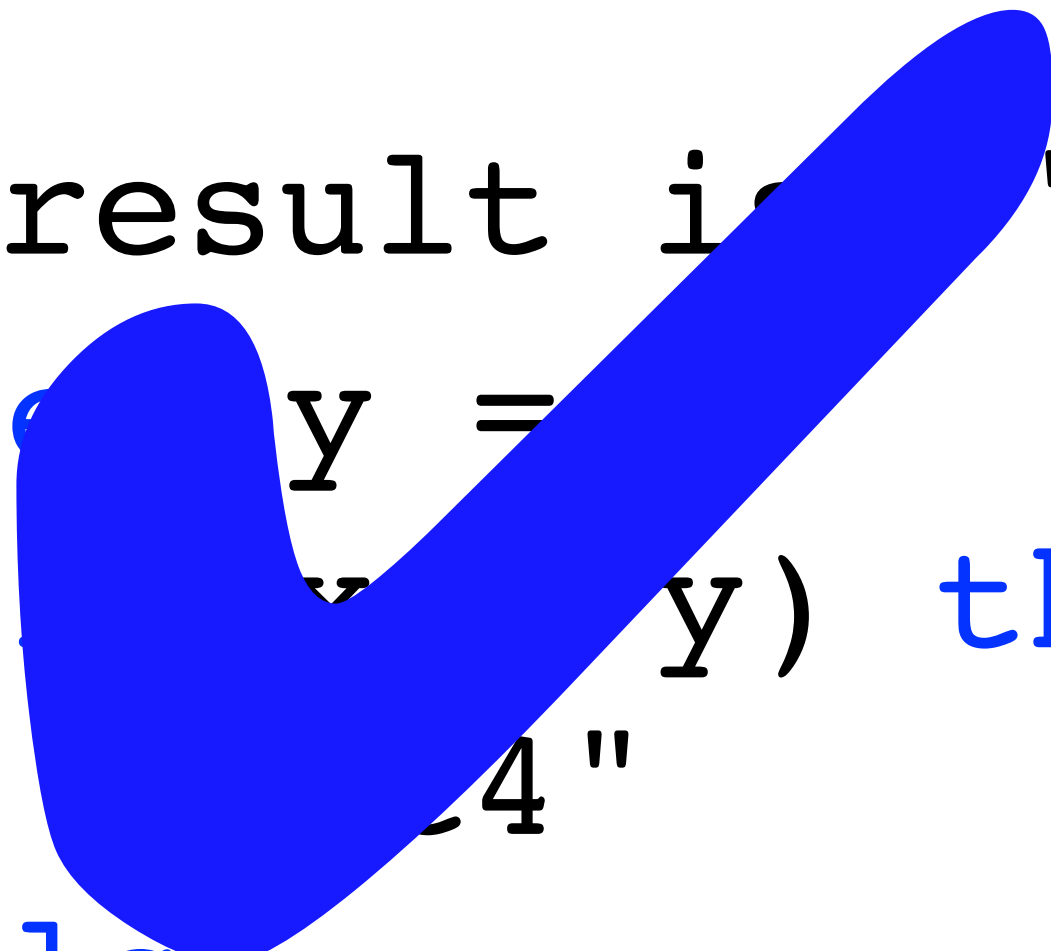
Java

```
"result is: "  
y  
if (y > 4)  
    return "gt4";  
}  
else  
    return "lt4";  
}
```



F#

```
"result is: " +  
let y =  
if (y > 4) then  
    "gt4"  
else  
    "lt4"
```



Statements and expressions

if-expressions in functional languages are
more like the ternary operator
(`<boolean> ? <val1> : <val2>`) in Java

Java

```
y = 4;  
return ("result is: " +  
    (x > y ? "gt4" : "lt4"));
```

F#

```
"result is: " +  
let y = 4  
if (x > y) then  
    "gt4"  
else  
    "lt4"
```

Some observations

As we will see, many things can be part of expressions

- Function declarations
- if-statements
- Pattern matches
- Arithmetic calculations
- ...

Questions?

Part II

Functions

Anonymous functions

Functions can be used as values or parameters to other functions just like standard integers or booleans

Anonymous functions

Functions can be used as values or parameters to other functions just like standard integers or booleans

There is NO difference!

Learn to appreciate this

Anonymous functions

Simple function expressions

```
fun r -> System.Math.PI * r * r
```

returns a function that has type
float -> float

Currying

Recall

A **function with multiple arguments** is actually a function with a single argument that **returns another function**

```
fun x y -> x + y
```

This is a function of type
 $\text{int} \rightarrow \text{int} \rightarrow \text{int}$

Currying

```
fun x y -> x + y
```

This is shorthand for:

```
fun x -> fun y -> x + y
```

This is a function of type
 $\text{int} \rightarrow \text{int} \rightarrow \text{int}$

Currying

```
fun x y -> x + y
```

This is shorthand for:

```
fun x -> (fun y -> x + y)
```

This is a function of type
`int -> (int -> int)`

Implicitly, we have
parentheses here

Partial Application

```
fun x -> fun y -> x + y
```

Partial Application

```
(fun x -> fun y -> x + y) 5
```


Partial Application

`(fun x -> fun y -> x + y) 5`



evaluates to

`fun y -> 5 + y`

Partial Application

`(fun x -> fun y -> x + y) 5`



evaluates to

`fun y -> 5 + y`

`fun x -> fun y -> x + y` has type `int -> (int -> int)`

Partial Application

`(fun x -> fun y -> x + y) 5`



evaluates to

`fun y -> 5 + y`

`fun x -> fun y -> x + y` has type `int -> (int -> int)`

`fun y -> 5 + y` has type `int -> int`

Function declarations

Assigning values to variables

```
let a = 5  
let b = true  
let c = 'Q'  
let d = (5, "meters")  
let e = 2.71828  
let f = fun x -> x + 3
```

Function declarations

`let f x = e`

means

`let f = fun x -> e`

Function declarations

For example:

```
let f x = x + 3
```

means

```
let f = fun x -> x + 3
```

Function declarations

Conceptually NO different to other declarations

```
let a = 5
let b = true
let c = 'Q'
let d = (5, "meters")
let e = 2.71828
let f = fun x -> x + 3
let g x = x + 3
```


Partial application

Suppose we have a cube with side length s (in meters), containing a liquid with density ρ (measured in kilograms per cube meter). The weight of the liquid is then given by

$$\rho * s^3$$

```
let weight rho s = rho * (s ** 3.0)
```

This function has the type
`float -> float -> float`

Partial application

```
let weight rho s = rho * (s ** 3.0)
```

```
let methanolWeight = weight 786.5;;  
val methanolWeight : float -> float
```

```
let waterWeight = weight 1000.0;;  
val waterWeight : float -> float
```

Partial application

```
let weight rho s = rho * (s ** 3.0)  
let methanolWeight = weight 786.5;;  
val methanolWeight : float -> float
```

One and Two cubic meters of methanol
then weigh respectively

```
methanolWeight 1.0  
val it : float = 786.5  
methanolWeight 2.0  
val it : float = 6292.0  
kilograms
```


Infix operators

Infix operators have prefix versions and are curried!

E.g. $+$ is an infix operator, i.e. we can write $5 + 6$

Infix operators

Infix operators have prefix versions and are curried!

E.g. $+$ is an infix operator, i.e. we can write $5 + 6$
 $(+)$ is the corresponding function for $+$

Infix operators

Infix operators have prefix versions and are curried!

E.g. $+$ is an infix operator, i.e. we can write $5 + 6$

$(+)$ is the corresponding function for $+$

$(+) \ 5 \ 6$ is the same as $5 + 6$

Infix operators

Infix operators have prefix versions and are curried!

E.g. $+$ is an infix operator, i.e. we can write $5 + 6$
 $(+)$ is the corresponding function for $+$

$(+) \ 5 \ 6$ is the same as $5 + 6$

$(+)$ is short for $\text{fun } x \ y \rightarrow x + y$

Infix operators

Infix operators have prefix versions and are curried

```
(+)  : int -> int -> int  
(-)  : int -> int -> int  
(*)  : int -> int -> int  
(/)  : int -> int -> int
```

These all have overloaded versions for other numeric types.

Infix operators

Infix operators can be partially applied

$(+) \ 5$

is the same as

`fun x -> 5 + x`

Infix operators

You can define your own infix operators

```
let ( .+ . ) = fun x y -> x + y
```


Infix operators

You can define your own infix operators

```
let ( .+ . ) = fun x y -> x + y
```

or, equivalently: `let ( .+ . ) x y = x + y`

Infix operators

You can define your own infix operators

```
let ( .+ . ) = fun x y -> x + y
```

or, equivalently: `let ( .+ . ) x y = x + y`

Now we can write: `5 .+ . 6`

Infix operators

You can define your own infix operators

```
let ( .+ . ) = fun x y -> x + y
```

or, equivalently: `let ( .+ . ) x y = x + y`

You will get to define several of these for the next assignment

Now we can write: `5 .+ . 6`

Patterns

We have seen how to use patterns when creating functions

`function`

[]	->	"Empty"
[x]	->	"One"
[x; y]	->	"Two"
[x; y; z]	->	"Three"
_	->	"Many"

Patterns

We have seen how to use patterns

Matches lists of the form $[v_1; v_2]$
x is assigned v_1
y is assigned v_2

function

[]	->	"Empty"
[x]	->	"One"
[x; y]	->	"Two"
[x; y; z]	->	"Three"
_	->	"Many"

Patterns

We have seen how to use patterns

```
let f = function
```

```
| []          -> "Empty"  
| [x]         -> "One"  
| [x; y]      -> "Two"  
| [x; y; z]   -> "Three"  
| _           -> "Many"
```

Matches lists of the form $[v_1; v_2]$
x is assigned v_1
y is assigned v_2

Patterns

We have seen how to use patterns

Matches lists of the form $[v_1; v_2]$
x is assigned v_1
y is assigned v_2

```
let f = function
```

```
| []      -> "Empty"  
| [x]     -> "One"  
| [x; y]  -> "Two"  
| [x; y; z] -> "Three"  
| _       -> "Many"
```

```
> f [1; 3];;  
val it : string = "Two"
```

Patterns

Patterns can also be used on their own, without creating a function

```
match lst with
| []          -> "Empty"
| [x]         -> "One"
| [x; y]      -> "Two"
| [x; y; z]   -> "Three"
| _           -> "Many"
```

Assuming the value `lst` is a list, this expression returns a string

Patterns

We can write `_` instead of a variable that we don't use on the right-hand side.

```
match lst with
| []          -> "Empty"
| [_]         -> "One"
| [_; _]      -> "Two"
| [_; _; _]   -> "Three"
| _           -> "Many"
```


Function vs fun + match

These two function expressions are identical:

function

[]	->	"Empty"
[_]	->	"One"
[_; _]	->	"Two"
[_; _; _]	->	"Three"
_	->	"Many"

fun lst ->

match lst with

[]	->	"Empty"
[_]	->	"One"
[_; _]	->	"Two"
[_; _; _]	->	"Three"
_	->	"Many"

Pattern matching = if on steroids

```
match x > 1 with  
| true  -> "Bigger"  
| false -> "Smaller"
```

is the same as

```
if x > 1  
  then "Bigger"  
  else "Smaller"
```

Nested let-statements

Nested let-statements are useful for local functions

```
let power x n =  
  let rec aux n =  
    match n with  
    | 0 -> 1.0  
    | m ->  
      x * aux (m - 1)  
  aux n
```

Nested let-statements

Nested let-statements are useful for local functions

```
let power x n =
  let rec aux n =
    match n with
    | 0 -> 1.0
    | m ->
      x * aux (m - 1)
  aux n
```

```
let power x n =
  let rec aux =
    function
    | 0 -> 1.0
    | m ->
      x * aux (m - 1)
  aux n
```

Same definition using
function instead.

Nested let-statements

Nested let-statements are useful for local functions

power 2.0 3

```
let power x n =  
  let rec aux n =  
    match n with  
    | 0 -> 1.0  
    | m ->  
      x * aux (m - 1)  
  aux n
```

Nested let-statements

Nested let-statements are useful for local functions

power 2.0 3 $\leadsto$
aux 3

```
let power x n =
  let rec aux n =
    match n with
    | 0 -> 1.0
    | m ->
      x * aux (m - 1)
  aux n
```


Nested let-statements

Nested let-statements are useful for local functions

```
let power x n =
  let rec aux n =
    match n with
    | 0 -> 1.0
    | m ->
      x * aux (m - 1)
  aux n
```

```
power 2.0 3 ~>
aux 3 ~>
2.0 * aux (3 - 1)
```

Nested let-statements

Nested let-statements are useful for local functions

```
let power x n =
  let rec aux n =
    match n with
    | 0 -> 1.0
    | m ->
      x * aux (m - 1)
  aux n
```

```
power 2.0 3 ~>
aux 3 ~>
2.0 * aux (3 - 1) ~>
2.0 * aux 2
```

Nested let-statements

Nested let-statements are useful for local functions

```
let power x n =
  let rec aux n =
    match n with
    | 0 -> 1.0
    | m ->
      x * aux (m - 1)
  aux n
```

```
power 2.0 3 ~>
aux 3 ~>
2.0 * aux (3 - 1) ~>
2.0 * aux 2 ~>
2.0 * (2.0 * aux (2 - 1))
```


Nested let-statements

Nested let-statements are useful for local functions

```
let power x n =
  let rec aux n =
    match n with
    | 0 -> 1.0
    | m ->
      x * aux (m - 1)
  aux n
```

```
power 2.0 3 ~>
aux 3 ~>
2.0 * aux (3 - 1) ~>
2.0 * aux 2 ~>
2.0 * (2.0 * aux (2 - 1)) ~>
2.0 * (2.0 * aux 1)
```

Nested let-statements

Nested let-statements are useful for local functions

```
let power x n =
  let rec aux n =
    match n with
    | 0 -> 1.0
    | m ->
      x * aux (m - 1)
  aux n
```

```
power 2.0 3 ~>
aux 3 ~>
2.0 * aux (3 - 1) ~>
2.0 * aux 2 ~>
2.0 * (2.0 * aux (2 - 1)) ~>
2.0 * (2.0 * aux 1) ~>
2.0 * (2.0 * (2.0 * aux (1 - 1)))
```


Nested let-statements

Nested let-statements are useful for local functions

```
let power x n =
  let rec aux n =
    match n with
    | 0 -> 1.0
    | m ->
      x * aux (m - 1)
  in
  aux n
```

power 2.0 3 $\leadsto$

aux 3 $\leadsto$

2.0 * aux (3 - 1) $\leadsto$

2.0 * aux 2 $\leadsto$

2.0 * (2.0 * aux (2 - 1)) $\leadsto$

2.0 * (2.0 * aux 1) $\leadsto$

2.0 * (2.0 * (2.0 * aux (1 - 1))) $\leadsto$

2.0 * (2.0 * (2.0 * aux 0))

Nested let-statements

Nested let-statements are useful for local functions

```
let power x n =
  let rec aux n =
    match n with
    | 0 -> 1.0
    | m ->
      x * aux (m - 1)
  in
  aux n
```

power 2.0 3 $\leadsto$

aux 3 $\leadsto$

2.0 * aux (3 - 1) $\leadsto$

2.0 * aux 2 $\leadsto$

2.0 * (2.0 * aux (2 - 1)) $\leadsto$

2.0 * (2.0 * aux 1) $\leadsto$

2.0 * (2.0 * (2.0 * aux (1 - 1))) $\leadsto$

2.0 * (2.0 * (2.0 * aux 0)) $\leadsto$

2.0 * (2.0 * (2.0 * 1.0))

Nested let-statements

Nested let-statements are useful for local functions

```
let power x n =
  let rec aux n =
    match n with
    | 0 -> 1.0
    | m ->
      x * aux (m - 1)
  in
  aux n
```

power 2.0 3 $\leadsto$

aux 3 $\leadsto$

2.0 * aux (3 - 1) $\leadsto$

2.0 * aux 2 $\leadsto$

2.0 * (2.0 * aux (2 - 1)) $\leadsto$

2.0 * (2.0 * aux 1) $\leadsto$

2.0 * (2.0 * (2.0 * aux (1 - 1))) $\leadsto$

2.0 * (2.0 * (2.0 * aux 0)) $\leadsto$

2.0 * (2.0 * (2.0 * 1.0)) $\leadsto$

2.0 * (2.0 * 2.0)

Nested let-statements

Nested let-statements are useful for local functions

```
let power x n =
  let rec aux n =
    match n with
    | 0 -> 1.0
    | m ->
      x * aux (m - 1)
  in aux n
```

power 2.0 3 $\leadsto$

aux 3 $\leadsto$

2.0 * aux (3 - 1) $\leadsto$

2.0 * aux 2 $\leadsto$

2.0 * (2.0 * aux (2 - 1)) $\leadsto$

2.0 * (2.0 * aux 1) $\leadsto$

2.0 * (2.0 * (2.0 * aux (1 - 1))) $\leadsto$

2.0 * (2.0 * (2.0 * aux 0)) $\leadsto$

2.0 * (2.0 * (2.0 * 1.0)) $\leadsto$

2.0 * (2.0 * 2.0) $\leadsto$

2.0 * 4.0

Nested let-statements

Nested let-statements are useful for local functions

```
let power x n =
  let rec aux n =
    match n with
    | 0 -> 1.0
    | m ->
      x * aux (m - 1)
  in aux n
```

```
power 2.0 3 ~>
aux 3 ~>
2.0 * aux (3 - 1) ~>
2.0 * aux 2 ~>
2.0 * (2.0 * aux (2 - 1)) ~>
2.0 * (2.0 * aux 1) ~>
2.0 * (2.0 * (2.0 * aux (1 - 1))) ~>
2.0 * (2.0 * (2.0 * aux 0)) ~>
2.0 * (2.0 * (2.0 * 1.0)) ~>
2.0 * (2.0 * 2.0) ~>
2.0 * 4.0 ~>
8.0
```


Function composition

Functions can be composed
In mathematics we write

$$(f \circ g)(x) = f(g(x))$$

In F# we write

$$g \gg f$$

Function composition

For example

Assume $f(x) = x + 3$ and $g(y) = y * y$

$$(f \circ g)(z) = z * z + 3$$

In F# we can write

```
let h = fun y -> y*y >>  
        fun x -> x+3
```

$$h\ 0 = 3$$

$$h\ 4 = 19$$

$$h\ 20 = 403$$

Function composition

Functions can be composed
In mathematics we write

$$(f \circ g)(x) = f(g(x))$$

In F# we write

$$g \gg f$$

Function composition

Functions can be composed
In mathematics we write

$$(f \circ g)(x) = f(g(x))$$

In F# we write

$g \gg f$ or $f \ll g$

Function composition

Functions can be composed
In mathematics we write

$$(f \circ g)(x) = f(g(x))$$

In F# we write

$g \gg f$ or $f << g$

The arrows point towards the outermost function

Function composition

Functions can be composed
In mathematics we write

$$(f \circ g)(x) = f(g(x))$$

In F# we write

$g \gg f$ or $f \ll g$

i.e. the function that will run last

Function composition

Functions can be composed
In mathematics we write

$$(f \circ g)(x) = f(g(x))$$

In F# we write

$g \gg f$ or $f \ll g$

```
let (>>) g f = fun x -> f (g x)
```

```
let (<<) f g = fun x -> f (g x)
```

Operators $|>$ and $<|$

The operator $|>$ means “send the value as argument to the function on the right”

$x \mid> f$ is equivalent to $f \ x$

The operator $<|$ means “send the value as argument to the function on the left”

$f \ <| \ x$ is equivalent to $f \ x$

Operators $|>$ and $<|$

The operator $|>$ means “send the value as argument to the function on the right”

This just seems like wasted effort... $x |> f$
is harder to read than $f x$

$f <| x$ is equivalent to $f x$

Operators $|>$ and $<|$

Both operators can be composed
(but we usually use $|>$ for that)

4 $|>$ `fun` $x \rightarrow x*x$ $|>$ $(+)$ 3 = 19

Operators $|>$ and $<|$

Both operators can be composed
(but we usually use $|>$ for that)

4 $|>$ `fun x -> x*x` $|>$ $(+)$ 3 = 19

Remember that

$(+)$ 3

is the same as

`fun x -> 3 + x`

Operators $|>$ and $<|$

While

$x \mid > f$

is not easier to read than

$f \ x,$

$x \mid > f \mid > g \mid > h$

is easier to read than

$h \ (g \ (f \ x))$

(especially when the functions are large or partially applied with many arguments) and naturally shows how x is transformed by the functions

Questions?

Part III

Lists & Polymorphism

Lists

We will expand on lists

- Recursion on lists
- Polymorphism

List constructors

Recall that lists are generated as follows

- `[]` is the **empty list**

List constructors

Recall that lists are generated as follows

- `[]` is the **empty list**
- `x :: xs` is a list with **head** `x` and **tail** `xs`

List constructors

Recall that lists are generated as follows

- `[]` is the **empty list**
- `x :: xs` is a list with **head** `x` and **tail** `xs`

In addition, `[x1; x2; ... ; xn]` is short for
`x1 :: (x2 :: ... (xn :: []))`

List constructors

Recall that lists are generated as follows

- `[]` is the **empty list**
- `x :: xs` is a list with **head** `x` and **tail** `xs`

In addition, `[x1; x2; ... ; xn]` is short for
`x1 :: (x2 :: ... (xn :: []))`

`[5; 3; -23]` is short for `5 :: (3 :: (-23 :: []))`

List Type

Lists have the type `t list`

Here, `t` is the type of the elements of the list.

E.g., `[5; 3; -23]` has type `int list`

List Type

Lists have the type `t list`

Here, `t` is the type of the elements of the list.

E.g., `[5; 3; -23]` has type `int list`

```
> 5::3::-23::[];;  
val it : int list = [5; 3; -23]
```

Length of a list

We recurse over the constructors

```
let rec length ls =  
  match ls with  
  | [] -> 0  
  | x::xs -> 1 + length xs
```

Length of a list

```
let rec length ls =  
  match ls with  
  | [] -> 0  
  | x::xs -> 1 + length xs
```


Length of a list

```
let rec length ls =  
  match ls with  
  | [] -> 0  
  | x::xs -> 1 + length xs  
  
length [1;2;3]
```

Length of a list

```
let rec length ls =  
  match ls with  
  | [] -> 0  
  | x::xs -> 1 + length xs  
  
length [1;2;3] = length (1 :: (2 :: (3 :: [])))
```


Length of a list

```
let rec length ls =
```

```
  match ls with
```

```
  | [] -> 0
```

```
  | x::xs -> 1 + length xs
```

```
length [1;2;3] = length (x1 :: (2 :: (3 :: [])))
```

Length of a list

```
let rec length ls =
```

```
  match ls with
```

```
  | [] -> 0
```

```
  | x::xs -> 1 + length xs
```

$\text{length } [1;2;3] = \text{length } (\overset{x}{\boxed{1}} :: \overset{xs}{\boxed{(2 :: (3 :: []))}})$

Length of a list

```
let rec length ls =
```

```
  match ls with
```

```
  | [] -> 0
```

```
  | x::xs -> 1 + length xs
```

$\text{length } [1;2;3] = \text{length } \overset{x}{(1)} :: \overset{xs}{(2 :: (3 :: []))}$
 $\leadsto 1 + \text{length } (2 :: (3 :: []))$

Length of a list

```
let rec length ls =  
  match ls with  
  | [] -> 0  
  | x::xs -> 1 + length xs
```

```
length [1;2;3] = length (1 :: (2 :: (3 :: [])))  
               ~> 1 + length (2 :: (3 :: []))  
                           x
```

Length of a list

```
let rec length ls =  
  match ls with  
  | [] -> 0  
  | x::xs -> 1 + length xs
```

```
length [1;2;3] = length (1 :: (2 :: (3 :: [])))  
    ~ 1 + length (2 :: (3 :: []))  
                  x      xs
```


Length of a list

```
let rec length ls =  
  match ls with  
  | [] -> 0  
  | x::xs -> 1 + length xs
```

```
length [1;2;3] = length (1 :: (2 :: (3 :: [])))  
  ~> 1 + length (2 :: (3 :: []))  
                        x      xs  
  ~> 1 + (1 + length (3 :: []))
```

Length of a list

```
let rec length ls =  
  match ls with  
  | [] -> 0  
  | x::xs -> 1 + length xs
```

```
length [1;2;3] = length (1 :: (2 :: (3 :: [])))  
               ~> 1 + length (2 :: (3 :: []))  
               ~> 1 + (1 + length (3 :: []))  
               ~> 1 + (1 + (1 + (length [])))
```

Length of a list

```
let rec length ls =  
  match ls with  
  | [] -> 0  
  | x::xs -> 1 + length xs
```

```
length [1;2;3] = length (1 :: (2 :: (3 :: [])))  
               ~> 1 + length (2 :: (3 :: []))  
               ~> 1 + (1 + length (3 :: []))  
               ~> 1 + (1 + (1 + (length [])))  
               ~> 1 + (1 + (1 + 0))
```


Length of a list

```
let rec length ls =  
  match ls with  
  | [] -> 0  
  | x::xs -> 1 + length xs
```

```
length [1;2;3] = length (1 :: (2 :: (3 :: [])))  
  ~> 1 + length (2 :: (3 :: []))  
  ~> 1 + (1 + length (3 :: []))  
  ~> 1 + (1 + (1 + (length [])))  
  ~> 1 + (1 + (1 + 0)) ~> 3
```

Polymorphic types

```
let rec length ls =  
  match ls with  
  | [] -> 0  
  | x::xs -> 1 + length xs
```


Polymorphic types

```
let rec length ls =  
  match ls with  
  | [] -> 0  
  | x::xs -> 1 + length xs
```

length has type

$\alpha \text{ 'list} \rightarrow \text{int}$

Polymorphic types

```
let rec length ls =  
  match ls with  
  | [] -> 0  
  | x::xs -> 1 + length xs
```

length has type

$a' \text{ list} \rightarrow \text{int}$

That is, the type of the elements of the list does not matter.

Polymorphic types

```
let rec length ls =  
  match ls with  
  | [] -> 0  
  | x::xs -> 1 + length xs
```

length has type

$a' \text{ list} \rightarrow \text{int}$

```
> length [1;2;3];;  
val it : int = 3
```

That is, the type of the elements of the list does not matter.

Polymorphic types

```
let rec length ls =  
  match ls with  
  | [] -> 0  
  | x::xs -> 1 + length xs
```

length has type

a' list -> int

That is, the type of the elements of the list does not matter.

```
> length [1;2;3];;  
val it : int = 3  
  
> length ['a';'b';'c'];;  
val it : int = 3
```

Polymorphic types

```
let rec length ls =  
  match ls with  
  | [] -> 0  
  | x::xs -> 1 + length xs
```

length has type

a' list -> int

That is, the type of the elements of the list does not matter.

```
> length [1;2;3];;  
val it : int = 3
```

```
> length ['a';'b';'c'];;  
val it : int = 3
```

```
> length ['a';2;'c'];;  
Type error!
```

Polymorphic types

```
let rec length ls =  
  match ls with  
  | [] -> 0  
  | x::xs -> 1 + length xs
```

length has type

a' list -> int

That is, the type of the elements of the list does not matter.

```
> length [1;2;3];;  
val it : int = 3
```

```
> length ['a';'b';'c'];;  
val it : int = 3
```

```
> length ['a';2;'c'];;  
Type error!
```

We cannot give a type to this list.

The type of list constructors

- `[]` has type `a' list`
- `::` has type `a' -> a' list -> a' list`

The type of list constructors

- `[]` has type `a' list`
- `::` has type `a' -> a' list -> a' list`

```
> let mylist = 5::3::-23::[];;  
val mylist : int list = [5; 3; -23]
```

The type of list constructors

- `[]` has type `a' list`
- `::` has type `a' -> a' list -> a' list`

```
> let mylist = 5::3::-23::[];;  
val mylist : int list = [5; 3; -23]
```

```
> 'c' :: mylist;;
```

Type mismatch.

Expecting a 'char list' but given a 'int list'

Appending lists

```
let rec (@) xs ys =  
  match xs with  
  | []      -> ys  
  | x::xs'  -> x :: (xs' @ ys)
```

Appending lists

```
let rec (@) xs ys =  
  match xs with  
  | []      -> ys  
  | x::xs'  -> x :: (xs' @ ys)
```

(@) has type $a' \text{ list} \rightarrow a' \text{ list} \rightarrow a' \text{ list}$

Appending lists

```
let rec (@) xs ys =
  match xs with
  | []      -> ys
  | x::xs'  -> x :: (xs' @ ys)
```

That is, the elements of the two lists must have the same type.

`(@)` has type $a' \text{ list} \rightarrow a' \text{ list} \rightarrow a' \text{ list}$

Appending lists

```
let rec (@) xs ys =
  match xs with
  | []      -> ys
  | x::xs'  -> x :: (xs' @ ys)
```

That is, the elements of the two lists must have the same type.

`(@)` has type $a' \text{ list} \rightarrow a' \text{ list} \rightarrow a' \text{ list}$

```
> [1;2] @ [3;4];;
val it : int list = [1; 2; 3; 4]
```

Appending lists

```
let rec (@) xs ys =
  match xs with
  | []      -> ys
  | x::xs'  -> x :: (xs' @ ys)
```

That is, the elements of the two lists must have the same type.

`(@)` has type $a' \text{ list} \rightarrow a' \text{ list} \rightarrow a' \text{ list}$

```
> [1;2] @ [3;4];;
val it : int list = [1; 2; 3; 4]
```

```
> [1;2] @ ['a'; 'b'];;
```

Type error!

Questions?

Part IV

Simple Type Definitions

Tuples

An ordered collection of n values $(v_1, v_2, \dots, v_n)$ is called an **n-tuple**.

```
> (5, 3, 2);;
```

```
val it : int * int * int = (5, 3, 2)
```

```
> (System.Math.PI, false)
```

```
val it : float * bool = (3.141592654, false)
```

```
> ("A function!!!", fun x -> x + 1)
```

```
val it : string * (int -> int) = ...
```

let-patterns

- Patterns can be used on left-hand side in a let declaration
- This binds the identifiers in the pattern to the values

```
> let (x,n) = (3,2);;  
val x : int = 3  
val n : int = 2
```


Types

It is possible to declare your own types

As an example, recall that the formula for solving a second degree polynomial is:

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Where $b^2 - 4ac$ is called the determinant

Example: Quadratic Formula

A second degree polynomial has the form

$$ax^2 + bx + c$$

A solution has two possible values and both can be represented by tuples:

```
type sdp      = float * float * float
```

```
type solution = float * float
```

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Example: Quadratic Formula

A second degree polynomial has the form

$$ax^2 + bx + c$$

A solution has two possible values and both can be represented by tuples:

```
type sdp      = float * float * float
```

```
type solution = float * float
```

We will use exceptions to handle division by zero and negative square roots.

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Example: Quadratic Formula

```
type sdp      = float * float * float
type solution = float * float

let solve (a,b,c) =
  if b * b - 4.0 * a * c < 0.0 || a = 0.0 then
    failwith "no solution"
  else
    ( (-b + sqrt(b * b - 4.0 * a * c)) / (2.0 * a),
      (-b - sqrt(b * b - 4.0 * a * c)) / (2.0 * a) )
```

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Example: Quadratic Formula

```

type sdp      = float * float * float
type solution = float * float

let solve (a,b,c) =
  if b * b - 4.0 * a * c < 0.0 || a = 0.0 then
    failwith "no solution"
  else
    ( (-b + sqrt(b * b - 4.0 * a * c)) / (2.0 * a),
      (-b - sqrt(b * b - 4.0 * a * c)) / (2.0 * a) )

```

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad \text{solve has type sdp} \rightarrow \text{solution}$$

Example: Quadratic Formula

```
type sdp      = float * float * float
```

```
type solution = float * float
```

```
let solve (a,b,c) =
```

```
  if b * b - 4.0 * a * c < 0.0 || a = 0.0 then
```

```
    failwith "no solution"
```

```
  else
```

```
    ( (-b + sqrt (b * b - 4.0 * a * c)) / (2.0 * a),  
      (-b - sqrt (b * b - 4.0 * a * c)) / (2.0 * a) )
```

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad \text{solve has type sdp} \rightarrow \text{solution}$$

Example: Quadratic Formula

```
type sdp      = float * float * float
```

```
type solution = float * float
```

```
let solve (a,b,c) =
```

```
  if b * b - 4.0 * a * c < 0.0 || a = 0.0 then
```

```
    failwith "no solution"
```

```
  else
```

```
    ( [-b + sqrt(b * b - 4.0 * a * c)] / (2.0 * a),  
      [-b - sqrt(b * b - 4.0 * a * c)] / (2.0 * a) )
```

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

solve has type sdp -> solution

Example: Quadratic Formula

```
type sdp      = float * float * float
```

```
type solution = float * float
```

```
let solve (a,b,c) =
```

```
  if b * b - 4.0 * a * c < 0.0 || a = 0.0 then
```

```
    failwith "no solution"
```

```
  else
```

```
    ( (-b + sqrt(b * b - 4.0 * a * c)) / (2.0 * a),  
      (-b - sqrt(b * b - 4.0 * a * c)) / (2.0 * a) )
```

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

solve has type sdp -> solution

Example: Quadratic Formula

```
type sdp      = float * float * float
```

```
type solution = float * float
```

```
let solve (a,b,c) =
```

```
  if b * b - 4.0 * a * c < 0.0 || a = 0.0 then
```

```
    failwith "no solution"
```

```
  else
```

```
    ( (-b + sqrt(b * b - 4.0 * a * c)) / (2.0 * a),  
      (-b - sqrt(b * b - 4.0 * a * c)) / (2.0 * a) )
```

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

solve has type sdp -> solution

Example: Quadratic Formula

```
type sdp      = float * float * float
```

```
type solution = float * float
```

What's the problem with this solution?

```
let solve (a,b,c) =
```

```
  if b * b - 4.0 * a * c < 0.0 || a = 0.0 then
```

```
    failwith "no solution"
```

```
  else
```

```
    ( (-b + sqrt(b * b - 4.0 * a * c)) / (2.0 * a),
```

```
      (-b - sqrt(b * b - 4.0 * a * c)) / (2.0 * a) )
```

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

solve has type sdp -> solution

Example: Quadratic Formula

```

type sdp      = float * float * float
type solution = float * float

let solve (a,b,c) =
  let d = b * b - 4.0 * a * c
  if d < 0.0 || a = 0.0 then failwith "no solution"
  else
    ( (-b + sqrt d) / (2.0 * a),
      (-b - sqrt d) / (2.0 * a) )

```

Nested let-statements let us reuse results from previous computations

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad \text{solve has type sdp} \rightarrow \text{solution}$$

Example: Quadratic Formula

```
type sdp      = float * float * float
```

```
type solution = float * float
```

```
let solve (a,b,c) =
```

```
  let d = b * b - 4.0 * a * c
```

```
  if d < 0.0 || a = 0.0 then failwith "no solution"
```

```
  else
```

```
    ( (-b + sqrt d) / (2.0 * a),
```

```
      (-b - sqrt d) / (2.0 * a) )
```

Nested let-statements let us reuse results from previous computations

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

solve has type sdp -> solution

Example: Quadratic Formula

```
type sdp      = float * float * float
```

```
type solution = float * float
```

```
let solve (a,b,c) =
```

```
  let sqrtD =
```

```
    let d = b * b - 4.0 * a * c
```

```
    if d < 0.0 || a = 0.0
```

```
    then failwith "no solution" else sqrt d
```

```
  ((-b + sqrtD) / (2.0 * a),
```

```
   (-b - sqrtD) / (2.0 * a))
```

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

solve has type sdp -> solution

let-statements can have
multi-level nesting

Records

Records allow us to structure data

```
type person =  
  { firstname : string;  
    lastname  : string;  
    age       : int }
```

Records are **similar to tuples**.
But in a record **each element has a label**.

Records

Records are declared by providing input to
all of their fields
(no partial application)

```
let jesper =  
  { firstname = "Jesper";  
    lastname = "Bengtson";  
    age = 41 } ;;  
val jesper : person =  
  { firstname = "Jesper";  
    lastname = "Bengtson";  
    age = 41 ; }
```


Records

Individual fields are accessed using their name

```
jesper.firstname;;  
val it : string = "Jesper"
```

```
jesper.lastname;;  
val it : string = "Bengtson"
```

```
jesper.age;;  
val it : int = 41
```

let-patterns

It is possible to obtain values from compound values

```
let { firstname = fn;  
    lastname = ln;  
    age = x } = jesper in  
    (fn, ln, x);;  
val it : string * string * int =  
    ("Jesper", "Bengtson", 41)
```

Tagged Values

We can define a type of **tagged values** as follows:

```
type Shape =  
  | Circle of float  
  | Square of float  
  | Triangle of float * float * float
```

This is also referred to as an **algebraic data type**.

Tagged Values

```
type Shape =  
  | Circle of float  
  | Square of float  
  | Triangle of float * float * float
```

```
> Circle 1.2;;  
val it : Shape = Circle 1.2  
  
> Triangle (1.5, 2.5; 1.0);;  
val it : Shape = Triangle (1.5, 2.5; 1.0)
```

Pattern Matching

```
type Shape =  
  | Circle of float  
  | Square of float  
  | Triangle of float * float * float
```

```
let area shape =  
  match shape with  
  | Circle r      -> System.Math.PI * r * r  
  | Square a      -> a * a  
  | Triangle(a,b,c) ->  
    let s = (a + b + c)/2.0  
    sqrt(s*(s-a)*(s-b)*(s-c))
```

Enumeration Type

Enumeration types are a special case of algebraic data types, where **constructors have no arguments**.

```
type Colour =  
  | Red  
  | Blue  
  | Green  
  | Yellow  
  | Purple
```


Enumeration Type

Enumeration types are a special case of algebraic data types, where **constructors have no arguments**.

```
type Colour =  
  | Red  
  | Blue  
  | Green  
  | Yellow  
  | Purple
```

```
let colourName c =  
  match c with  
  | Red    -> "red"  
  | Blue   -> "blue"  
  | Green  -> "green"  
  | Yellow -> "yellow"  
  | Purple -> "purple"
```

Questions?